
Procedimientos Almacenados y Funciones en MySQL

Introducción

Las bases de datos modernas no sólo almacenan datos, sino que también pueden contener lógica de programación que se ejecuta directamente en el servidor de base de datos. En MySQL, esta capacidad se implementa principalmente a través de dos estructuras: los procedimientos almacenados y las funciones. Estos elementos forman parte de lo que se conoce como “programación del lado del servidor” y representan una parte fundamental en el diseño de sistemas de bases de datos robustos y eficientes.

1. Procedimientos Almacenados

1.1 ¿Qué es un Procedimiento Almacenado?

Un procedimiento almacenado es un conjunto de instrucciones SQL que se almacenan en el servidor de base de datos y pueden ser invocados mediante un nombre específico. Estos procedimientos pueden recibir parámetros, ejecutar operaciones complejas y devolver múltiples resultados.

1.2 Ventajas de los Procedimientos Almacenados

- **Rendimiento mejorado:** Las consultas precompiladas reducen el tiempo de ejecución.
- **Seguridad:** Permiten controlar el acceso a las tablas subyacentes.
- **Reutilización de código:** Centraliza la lógica de negocio en la base de datos.
- **Reducción del tráfico de red:** Solo se envía la llamada al procedimiento, no todas las instrucciones SQL.
- **Mantenimiento:** Facilita la modificación de lógica sin cambiar el código de aplicación.
- **Transacciones:** Simplifican la gestión de transacciones complejas.

1.3 Sintaxis Básica

```
DELIMITER //
```

```
CREATE PROCEDURE nombre_procedimiento([parámetros])
```

```
BEGIN
```

```
    -- Instrucciones SQL
```

```
END //
```

```
DELIMITER ;
```

1.4 Tipos de Parámetros

En MySQL, los procedimientos almacenados pueden tener tres tipos de parámetros:

- **IN:** Parámetros de entrada (valor por defecto si no se especifica).
- **OUT:** Parámetros de salida que devuelven valores.
- **INOUT:** Parámetros que funcionan tanto para entrada como para salida.

```

DELIMITER //
CREATE PROCEDURE calcular_estadisticas(
    IN tabla_nombre VARCHAR(100),
    OUT total_registros INT,
    INOUT ultimo_registro DATETIME
)
BEGIN
    SET @query = CONCAT('SELECT COUNT(*) INTO @count FROM ', tabla_nombre);
    PREPARE stmt FROM @query;
    EXECUTE stmt;
    DEALLOCATE PREPARE stmt;

    SET total_registros = @count;

    SET @query = CONCAT('SELECT MAX(fecha_creacion) INTO @ultima_fecha FROM ', tabla_nombre);
    PREPARE stmt FROM @query;
    EXECUTE stmt;
    DEALLOCATE PREPARE stmt;

    SET ultimo_registro = @ultima_fecha;
END //
DELIMITER ;

```

1.5 Invocación de Procedimientos

Para ejecutar un procedimiento almacenado:

```

-- Sin parámetros
CALL nombre_procedimiento();

-- Con parámetros IN
CALL obtener_empleados('Ventas');

-- Con parámetros OUT
CALL calcular_total(id_cliente, @total);
SELECT @total AS total_calculado;

-- Con parámetros INOUT
SET @fecha = '2023-01-01';
CALL calcular_estadisticas('empleados', @total_emp, @fecha);
SELECT @total_emp, @fecha;

```

1.6 Variables en Procedimientos Almacenados

MySQL permite declarar y utilizar variables locales dentro de procedimientos:

```
DELIMITER //
```

```
CREATE PROCEDURE calcular_edad(IN fecha_nacimiento DATE, OUT edad INT)
```

```
BEGIN
```

```
    DECLARE fecha_actual DATE;
```

```
    SET fecha_actual = CURDATE();
```

```
    SET edad = YEAR(fecha_actual) - YEAR(fecha_nacimiento);
```

```
    -- Ajuste si todavía no ha cumplido años en el año actual
```

```
    IF DATE_FORMAT(fecha_nacimiento, '%m%d') > DATE_FORMAT(fecha_actual, '%m%d') THEN
```

```
        SET edad = edad - 1;
```

```
    END IF;
```

```
END //
```

```
DELIMITER ;
```

1.7 Estructuras de Control

Condicionales

```

-- IF-THEN-ELSE
IF condicion THEN
    -- instrucciones
ELSEIF otra_condicion THEN
    -- instrucciones
ELSE
    -- instrucciones
END IF;

-- CASE
CASE expresion
    WHEN valor1 THEN
        -- instrucciones
    WHEN valor2 THEN
        -- instrucciones
    ELSE
        -- instrucciones
END CASE;

```

Bucles

```

-- WHILE
WHILE condicion DO
    -- instrucciones
END WHILE;

-- REPEAT (ejecuta al menos una vez)
REPEAT
    -- instrucciones
UNTIL condicion
END REPEAT;

-- LOOP con LEAVE
etiqueta: LOOP
    -- instrucciones
    IF condicion THEN
        LEAVE etiqueta;
    END IF;
END LOOP etiqueta;

```

1.8 Manejo de Errores

MySQL proporciona un mecanismo para gestionar errores mediante handlers:

```

DELIMITER //
CREATE PROCEDURE insertar_cliente(
    IN p_nombre VARCHAR(100),
    IN p_email VARCHAR(100),
    OUT p_resultado VARCHAR(100)
)
BEGIN
    -- Declaración de handler para capturar errores
    DECLARE exit handler for SQLEXCEPTION
    BEGIN
        SET p_resultado = 'Error al insertar cliente';
        ROLLBACK;
    END;

    -- Iniciar transacción
    START TRANSACTION;

    INSERT INTO clientes(nombre, email) VALUES (p_nombre, p_email);

    SET p_resultado = 'Cliente insertado correctamente';

    COMMIT;
END //
DELIMITER ;

```

2. Funciones Almacenadas

2.1 ¿Qué es una Función Almacenada?

Una función almacenada es similar a un procedimiento, pero está diseñada para calcular y devolver un único valor. Se utilizan típicamente en expresiones SQL.

2.2 Diferencias entre Procedimientos y Funciones

Característica	Procedimiento	Función
<u>Valor retornado</u>	Puede retornar múltiples valores mediante parámetros OUT	Retorna un único valor
<u>Uso</u>	Se invoca con CALL	Se utiliza en expresiones SQL
<u>Transacciones</u>	Puede contener transacciones	No puede contener transacciones
<u>Sentencias permitidas</u>	Prácticamente todas	Limitadas (no permite sentencias que retornen conjuntos de resultados)

2.3 Sintaxis Básica

```
DELIMITER //
```

```
CREATE FUNCTION nombre_funcion([parámetros])
```

```
RETURNS tipo_dato
```

```
BEGIN
```

```
    -- Instrucciones SQL
```

```
    RETURN valor;
```

```
END //
```

```
DELIMITER ;
```

2.4 Ejemplos de Funciones

```

DELIMITER //
CREATE FUNCTION calcular_iva(precio DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN precio * 0.21;
END //
DELIMITER ;

-- Uso de la función
SELECT nombre, precio, calcular_iva(precio) AS iva,
       precio + calcular_iva(precio) AS precio_con_iva
FROM productos;

```

3. Gestión y Administración

3.1 Consultar Procedimientos y Funciones Existentes

-- Listar todos los procedimientos

```
SHOW PROCEDURE STATUS WHERE Db = 'nombre_base_datos';
```

-- Listar todas las funciones

```
SHOW FUNCTION STATUS WHERE Db = 'nombre_base_datos';
```

-- Ver la definición de un procedimiento

```
SHOW CREATE PROCEDURE nombre_procedimiento;
```

-- Ver la definición de una función

```
SHOW CREATE FUNCTION nombre_funcion;
```

3.2 Modificar Procedimientos y Funciones

En MySQL existe el comando ALTER PROCEDURE, que sirve para cambiar características del procedimiento, pero no permite cambiar parámetros o cuerpo del procedimiento. Para modificar un procedimiento o función, es necesario eliminarlo y volver a crearlo:

```
DROP PROCEDURE IF EXISTS nombre_procedimiento;
```

-- Crear nuevamente con la modificación

```
DROP FUNCTION IF EXISTS nombre_funcion;
```

-- Crear nuevamente con la modificación

3.3 Eliminar Procedimientos y Funciones

```
DROP PROCEDURE IF EXISTS nombre_procedimiento;  
DROP FUNCTION IF EXISTS nombre_funcion;
```

4. Buenas Prácticas

4.1 Convenciones de Nomenclatura

- Utilizar prefijos para identificar el tipo de objeto: sp_ para procedimientos, fn_ para funciones.
- Nombres descriptivos que indiquen la acción que realizan.
- Usar verbos en infinitivo para procedimientos (ej. sp_crear_cliente).
- Usar sustantivos o adjetivos para funciones (ej. fn_edad_cliente).

4.2 Documentación Interna

Incluir comentarios al inicio del procedimiento o función:

```

DELIMITER //
CREATE PROCEDURE sp_actualizar_cliente(
    IN p_id INT,
    IN p_nombre VARCHAR(100),
    IN p_email VARCHAR(100),
    OUT p_resultado VARCHAR(100)
)
BEGIN
    /*
    Autor: Nombre del Desarrollador
    Fecha: 2023-03-15
    Descripción: Actualiza la información de un cliente existente
    Parámetros:
        p_id - ID del cliente a actualizar
        p_nombre - Nuevo nombre del cliente
        p_email - Nuevo email del cliente
        p_resultado - Mensaje de resultado de la operación
    Modificaciones:
        2023-03-20 - Se agregó validación de email
    */

    -- Código del procedimiento
END //
DELIMITER ;

```

4.3 Seguridad

- Validar siempre los parámetros de entrada para prevenir inyección SQL.
- Utilizar prepared statements cuando se construyan consultas dinámicas.
- Implementar el principio de privilegio mínimo en los permisos.
- Encriptar datos sensibles antes de almacenarlos.

4.4 Rendimiento

- Mantener los procedimientos y funciones simples y enfocados en una tarea específica.
- Evitar el uso excesivo de cursores, preferir operaciones basadas en conjuntos.
- Optimizar consultas SQL dentro de los procedimientos.
- Usar índices apropiadamente en las tablas referenciadas.
- Considerar la posibilidad de particionar tablas grandes.

4.5 Gestión de Transacciones

- Utilizar transacciones para operaciones que modifican múltiples tablas.
- Asegurarse de que cada camino de ejecución termine con COMMIT o ROLLBACK.
- Implementar manejo de errores adecuado para garantizar la integridad de los datos.

5. Ejercicios Prácticos

5.1 Crear una Base de Datos de Ejemplo

```
CREATE DATABASE IF NOT EXISTS biblioteca;  
USE biblioteca;
```

```
CREATE TABLE autores (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  nacionalidad VARCHAR(50),  
  fecha_nacimiento DATE  
);
```

```
CREATE TABLE libros (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  titulo VARCHAR(200) NOT NULL,  
  autor_id INT,  
  genero VARCHAR(50),  
  anio_publicacion INT,  
  disponible BOOLEAN DEFAULT TRUE,  
  FOREIGN KEY (autor_id) REFERENCES autores(id)  
);
```

```
CREATE TABLE prestamos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  libro_id INT,  
  nombre_usuario VARCHAR(100),  
  fecha_prestamo DATE,  
  fecha_devolucion_prevista DATE,  
  fecha_devolucion_real DATE,  
  FOREIGN KEY (libro_id) REFERENCES libros(id)  
);
```

5.2 Ejercicio 1: Procedimiento para Préstamo de Libros

Crear un procedimiento almacenado que gestione el préstamo de un libro, verificando su disponibilidad y actualizando el estado.

5.3 Ejercicio 2: Función para Calcular Multas

Crear una función que calcule la multa por retraso en la devolución de un libro, basándose en los días de retraso.

5.4 Ejercicio 3: Procedimiento con Cursores

Crear un procedimiento que genere un reporte de los libros nunca prestados, utilizando un cursor para procesar cada libro.

5.5 Ejercicio 4: Trigger y Procedimiento

Crear un trigger que llame a un procedimiento almacenado cada vez que se registre una devolución, para actualizar estadísticas.

6. Conclusiones

Los procedimientos almacenados y funciones en MySQL proporcionan una poderosa forma de encapsular lógica de negocio dentro de la base de datos. Utilizados correctamente, ofrecen ventajas significativas en términos de rendimiento, seguridad y mantenibilidad.

Al dominar estas herramientas, un profesional de bases de datos puede diseñar soluciones más robustas y eficientes, reduciendo la carga en las aplicaciones cliente y centralizando la lógica crítica donde los datos residen.

La práctica constante y la aplicación de las buenas prácticas mencionadas en este documento permitirán aprovechar al máximo estas capacidades de MySQL, resultando en sistemas de información más confiables y escalables.

7. Referencias y Recursos Adicionales

- Documentación oficial de MySQL:
<https://dev.mysql.com/doc/refman/8.0/en/stored-programs.html>
- Libros recomendados:

- “High Performance MySQL” por Baron Schwartz, Peter Zaitsev y Vadim Tkachenko
 - “MySQL Stored Procedure Programming” por Guy Harrison y Steven Feuerstein
- Tutoriales en línea:
 - MySQL Tutorial: <https://www.mysqltutorial.org/mysql-stored-procedure/>
 - W3Schools MySQL:
https://www.w3schools.com/sql/sql_stored_procedures.asp